Title: Using Numerical Analysis Methods to Approximate Expected Premium

As Well as Making a Cubic Spline Model for Losses

By: Nathan Tassey, and Patrick Lakomy

Introduction:

Our project hinges on various numerical analysis concepts to (a) create a pricing/loss model for loss data, and (b) approximate the expected premium of all claims presented in our data. The main features of our project involves using three numerical approaches.

1. Using Newton's Root Finding Method to find σ, and Expected Losses
2. Construction a Cubic Spline for Losses based on a calculated Risk Score
3. Use Numerical Integration to find Expected Losses (2 ways)

Project Outline/steps:

Step 1. collect the data, analyze the loss distribution, and convert data to proper form.

Step 2. Create parameters to approximate using newton's method. Take those parameters to find the expected loss amount.

Step 3. define risk classes and dive further into premium estimation across each risk class. Create a cubic spline to (a) model the premium, and (b) through using numerical inegration find the expected value of this model.

Step 4. Use the lognormal density formula, and our loss values to find the expected value of our losses using numerical integration

Step 5. Compare the three methods to each other and the theoretical exact value.

Step 6. Conclusion

Step 1. The Data

In claims data like this set given by Allstate in their claims severity competition, you tend to see several variables like cont1, cat1, and plenty of other variables. These variables are anonymized variables that reveal data about individual people. cont1, cont2, cont(x) are continuous variables, and can take on any numerical value. For example, in an auto insurance data set this could be driver age, vehicle value, policyholder credit score, time insured, company created risk score, and so on. cat1, cat2, cat(x) are categorical variables, and they take on labels and groups, not numbers. For example this could mean state, vehicle type, gender, and so on. These variables have been replaced with cont(x) and cat(x) to anonymize the data, so you can see the values and categories that cat(x) values set specific policyholder id's into to draw conclusions on the average loss amount per id, not necessarily the interpretation of the data. You can draw insights and conclusions from data without knowing what the values represent. Another reason why the variables are anonymized is for privacy reasons. The real variables are from Allstates personal proprietary business information and contain highly sensitive information about their policyholders. This could include driving history, credit card number, zip code, and plenty of other things. Releasing the data would allow people reading the data to potentially identify individuals, and data analysts could use this information to reverse engineer Allstates underwriting strategy to compete against them. Since this dataset was created based off real information from Allstate, for their competition, the information is anonymized, but still categorized to allow you to draw conclusions from it. For example, cat3 = B tends to have a higher loss (this may or may not be true, just an example of a conclusion to be potentially drawn from this dataset.)

In summary, the dataset is anonymized which limits interpretation, but it allows us to draw real world conclusions from real world data. It's actually useful for data analysts to focus on variables that could potentially be statistically significant instead of the actual variable and what it represents. For example, maybe X age drivers get into more accidents and have a higher loss amount on average. The same conclusion could be drawn that cont3 = x is positively correlated with higher loss amounts and frequencies. cont(x) are numerical values. cat(x) are categorical values. loss is the loss amount for each id. id is the policyholder id and demonstrates a claim made to an insurance company by that number id.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

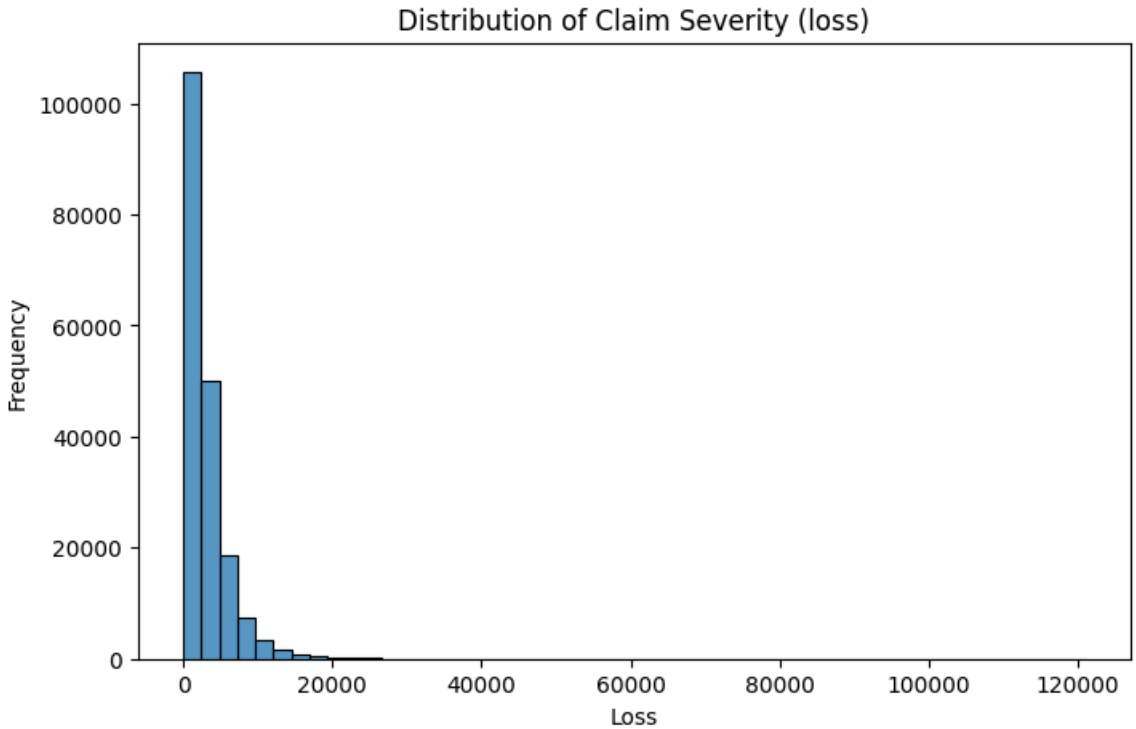
```python
import seaborn as sns

# Mount Drive
from google.colab import drive
drive.mount('/content/drive')

# Loading the data
df = pd.read_csv('/content/drive/MyDrive/NumAnalysis/Copy of train.csv')

# Previews
print(df.head())

print(df.info())

print(df.describe())
```

```
Mounted at /content/drive
   id cat1 cat2 cat3 cat4 cat5 cat6 cat7 cat8 cat9  ...     cont6     cont7  \
0   1    A    B    A    B    A    A    A    A    B  ...  0.718367  0.335060
1   2    A    B    A    A    A    A    A    A    B  ...  0.438917  0.436585
2   5    A    B    A    A    B    A    A    A    B  ...  0.289648  0.315545
3  10    B    B    A    B    A    A    A    A    B  ...  0.440945  0.391128
4  11    A    B    A    B    A    A    A    A    B  ...  0.178193  0.247408

     cont8    cont9   cont10    cont11    cont12    cont13    cont14     loss
0  0.30260  0.67135  0.83510  0.569745  0.594646  0.822493  0.714843  2213.18
1  0.60087  0.35127  0.43919  0.338312  0.366307  0.611431  0.304496  1283.60
2  0.27320  0.26076  0.32446  0.381398  0.373424  0.195709  0.774425  3005.09
3  0.31796  0.32128  0.44467  0.327915  0.321570  0.605077  0.602642   939.85
4  0.24564  0.22089  0.21230  0.204687  0.202213  0.246011  0.432606  2763.85

[5 rows x 132 columns]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 188318 entries, 0 to 188317
Columns: 132 entries, id to loss
dtypes: float64(15), int64(1), object(116)
memory usage: 189.7+ MB
None
                  id          cont1          cont2          cont3  \
count  188318.000000  188318.000000  188318.000000  188318.000000
mean   294135.982561       0.493861       0.507188       0.498918
std    169336.084867       0.187640       0.207202       0.202105
min         1.000000       0.000016       0.001149       0.002634
25%    147748.250000       0.346090       0.358319       0.336963
50%    294539.500000       0.475784       0.555782       0.527991
75%    440680.500000       0.623912       0.681761       0.634224
max    587633.000000       0.984975       0.862654       0.944251

               cont4          cont5          cont6          cont7  \
count  188318.000000  188318.000000  188318.000000  188318.000000
mean        0.491812       0.487428       0.490945       0.484970
std         0.211292       0.209027       0.205273       0.178450
min         0.176921       0.281143       0.012683       0.069503
25%         0.327354       0.281143       0.336105       0.350175
50%         0.452887       0.422268       0.440945       0.438285
75%         0.652072       0.643315       0.655021       0.591045
max         0.954297       0.983674       0.997162       1.000000

               cont8          cont9         cont10         cont11  \
count  188318.000000  188318.000000  188318.000000  188318.000000
mean        0.486437       0.485506       0.498066       0.493511
std         0.199370       0.181660       0.185877       0.209737
min         0.236880       0.000080       0.000000       0.035321
25%         0.312800       0.358970       0.364580       0.310961
50%         0.441060       0.441450       0.461190       0.457203
75%         0.623580       0.566820       0.614590       0.678924
max         0.980200       0.995400       0.994980       0.998742

              cont12         cont13         cont14           loss
count  188318.000000  188318.000000  188318.000000  188318.000000
mean        0.493150       0.493138       0.495717    3037.337686
std         0.209427       0.212777       0.222488    2904.086186
min         0.036232       0.000228       0.179722       0.670000
25%         0.311661       0.315758       0.294610    1204.460000
50%         0.462286       0.363547       0.407403    2115.570000
75%         0.675759       0.689974       0.724623    3864.045000
max         0.998484       0.988494       0.844848  121012.250000
```

Analyzing the Data

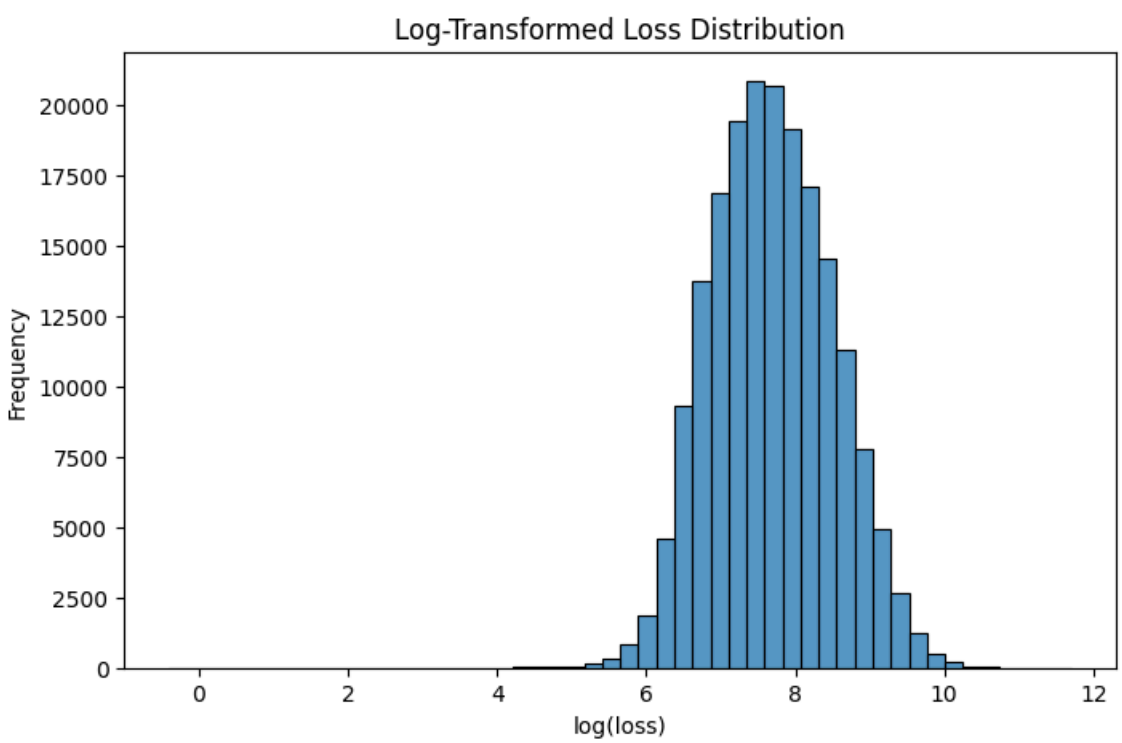
In [2]:

```
#Plotting the loss distribution

plt.figure(figsize=(8,5)) # plotting a 5 inch by 8 inch plot
sns.histplot(df['loss'], bins=50) # a histogram on seaborn using the loss data from our dataset, pulling out all the values a
plt.title('Distribution of Claim Severity (loss)') # title
plt.xlabel('Loss') # Loss on the x axis
plt.ylabel('Frequency') # what amount appears the most frequently
plt.show() # show plot
```



Here you see a huge spike at zero and a much longer right tail. This means that taking the log of all our values would be more valuable

In [3]:
```
plt.figure(figsize=(8,5)) # plot width 8 inches by 5 inches
sns.histplot(np.log(df['loss']), bins=50) # seaborn histogram but we take the log of the loss amount
plt.title('Log-Transformed Loss Distribution') # plot title
plt.xlabel('log(loss)') # x axis is loss amount but the logarithm of it to keep the spacing even
plt.ylabel('Frequency') # y axis is
plt.show()

# interpretation of the data
mean_loss = df['loss'].mean()
median_loss = df['loss'].median()
std_loss = df['loss'].std()

print('Mean:', mean_loss)
print('Median:', median_loss)
print('Standard Deviation:', std_loss)
```



```
Mean: 3037.3376856699833
Median: 2115.5699999999997
Standard Deviation: 2904.0861863904365
```

Step 2: Newton's Root Finding method

Now that we have a normal distribution (as seen by the bell curve), we know that log(loss) ~ Normal(μ,σ^2) (A.K.A., log(loss) is a random variable distributed normally with some mean and some variance sigma squared). Therefore, the loss distribution follows a lognormal distribution.

loss ~ Lognormal(μ, σ^2)

With this information we can use the expected loss formula for a lognormal random variable:

$E[\text{loss}] = e^{\mu + \frac{\sigma^2}{2}}$

Since our data follows a normal distribution, we can use our data to estimate μ and σ. With this information, we can verify σ using Newton's Method.

For a normal distribution, the MLE (Maximum Loss Expectancy, or the worst case financial loss from a given risk) for σ satisfies the following equation:

$f(\sigma) = \frac{1}{n} \sum \frac{(x_i - \mu)^2}{\sigma^2} - 1$

Where n is the number of data points where the loss amount is positive, since we are modeling *severity*, there is no severity when there is no loss amount.

For the normal model, variance already has a closed form solution and can be estimated without Newton's Method. However, we reformulate the closed-form variance estimator as a root-finding problem to demonstrate Newton's method on a known analytical solution.

In [4]:
```
# finding mu
log_loss = np.log(df[df['loss'] > 0]['loss'])
mu = log_loss.mean()
print('mu:', mu)

# Std. dev to compare to the root found std. dev
sigma_direct = log_loss.std()
print('sigma_direct:', sigma_direct)

# finding n in our data
positive_losses = df[df['loss'] > 0]
n = len(positive_losses)
print('n:', n)
```

```
mu: 7.685267842849341
sigma_direct: 0.8118680942954611
n: 188318
```

Where The Newton's Root Finding Method Fits in (Using the Newton's Method):

Since n is so massive, it would take us years to solve this manually. Instead, we can use python to calculate $\frac{1}{n}\sum(x_i - \mu)^2$, and set it to A. Then the problem becomes

$f(\sigma) = \frac{A}{\sigma^2} - 1,$

Where A is a constant.

We can now take the derivative of this function and make an initial guess of 1 to find the root of σ.

Where our function is: $f(\sigma) = \frac{A}{\sigma^2} - 1$

and our derivative is: $f'(\sigma) = -\frac{2A}{\sigma^3}$

Recall, the method states $\sigma_i = \sigma_{i-1} + \frac{f(\sigma_{i-1})}{f'(\sigma_{i-1})}$

Giving us; $\sigma_i = \sigma_{i-1} + \frac{\frac{A}{(\sigma_{i-1})^2} - 1}{\frac{2A}{(\sigma_{i-1})^3}}$

Finally we need an initial guess. Because we know σ must positive we can make a guess very very small, and very close to zero. We decided to set our initial guess at 0.0000001.

After the root is found/converges we plug it into our expected value function: $E[\text{loss}] = e^{\mu + \frac{\sigma^2}{2}}$

This gives us our first numerical approximation for the expected value of the losses.

Finding the A value for our function

In [5]:
```
A = np.mean((log_loss - mu)**2) # A is the sum of all log_loss variables minus the mean squared.
print('A: ', A)
```

```
A:  0.6591263024457225
```

Finding our 'root' using newton's method and using it to approimate our expected losses.

In [6]:
```
#newtons method for root finding

#define our variables
variable_dict = {'mean': mu, 'A': A, 'Root Guess': 0.0000001}
    #I made the root guess 0.0001 as we know the variance must be positive

#define symbols for the function
import sympy as sm
sigma = sm.Symbol('sigma', positive = True)

#define function(viewable for text)
f_sigma_sym = variable_dict['A']/sigma**2 - 1
f_sigma_prime_sym = sm.diff(f_sigma_sym, sigma)

#define actual functions(useable)
f_sigma = sm.lambdify(sigma, f_sigma_sym, 'numpy')
f_sigma_prime = sm.lambdify(sigma, f_sigma_prime_sym, 'numpy')

#run root finding method
from scipy.optimize import newton #found using the python documentation

#find root
root = newton(func=f_sigma, x0=variable_dict['Root Guess'], fprime=f_sigma_prime, maxiter=1000)
print('Found Root:', root)
```

```
Found Root: 0.8118659387150824
```

Our root using Newton's method becomes 0.8118659387150824

Now, since we have our root we are going to use the expected value formula mentioned before to approximate our expected premium/loss.

recall; $E[X] = e^{\mu + \frac{\sigma^2}{2}}$

Where $\sigma = root = 0.8118659387150824$

In [58]:
```
#finding expected loss
expected_loss = np.exp(mu + root**2/2)
print('Expected Loss:', expected_loss)
```

```
Expected Loss: 3025.498027404372
```

Method (1) yields: $E[X] = 3025.498027404372$

Step 3: Creating a Cubic Spline and Preforming Numerical Integration to Aproximate $E[X]$

Before we dive into the creation of our cubic spline model, we must analyze the data to determine the variable to be used as our independent variable, to predict our independent variable, which is losses. Having these two would give us a (x,y) mapping to create our cubic spline.

Step 3a: Variable Analysis

As mentioned, we need to find the proper variable to use as our independent variable. To achieve this, we have decided to do binning and graph analysis to find which variable has the greatest connection to our losses (dependent analysis). While steo 3a is not pure numerical analysis, it sets up the proper foundation for both creating a cubic spline and preforming numerical integration.

In [8]:
```
# attempting to build a relative frequency model for groups
# need to look at average loss amount between each categorical variable and look for trends
print(df.columns) # to see all the variables to potentially group by
```

```
Index(['id', 'cat1', 'cat2', 'cat3', 'cat4', 'cat5', 'cat6', 'cat7', 'cat8',
       'cat9',
       ...
       'cont6', 'cont7', 'cont8', 'cont9', 'cont10', 'cont11', 'cont12',
       'cont13', 'cont14', 'loss'],
      dtype='object', length=132)
```

In [9]:
```
group_cat1 = df.groupby('cat1')['loss'].agg(['count', 'mean', 'median', 'std']) # groupby splits the dataset into groups base
print(group_cat1)                                                               # this shows us the different groups cat1 has
```

```
       count         mean   median          std
cat1
A     141550  3408.091515  2458.69  3122.436629
B      46768  1915.198521  1444.86  1666.082337
```

Using economic/math priniples, we can see the ratio of the counts are about 3:1 which is good, and the mean of A is almost double the amount of B. This means cat1 is very predictive of loss. Since theres a higher mean, median, and std, we can already confirm that group A is more expensive and more volatile. We can conclude with some confidence that this variable separates risk, has a strong signal and has plenty of data. this means we have a good variable to break into frequency and severity and now the expected loss of the two groups vary. this gives us even more insight and more accuracy when estimating claims

main takeaway: policies in A have significantly higher expected losses and variability compared to B, indicating that cat1 is a strong predictor of risk and a good variable to group risk based off.

In [10]:
```
# testing cat2
group_cat2 = df.groupby('cat2')['loss'].agg(['count', 'mean', 'median', 'std'])
print(group_cat2)
```

```
       count         mean   median          std
cat2
A     106721  2454.139844  1739.25  2361.309860
B      81597  3800.104171  2833.99  3338.421975
```

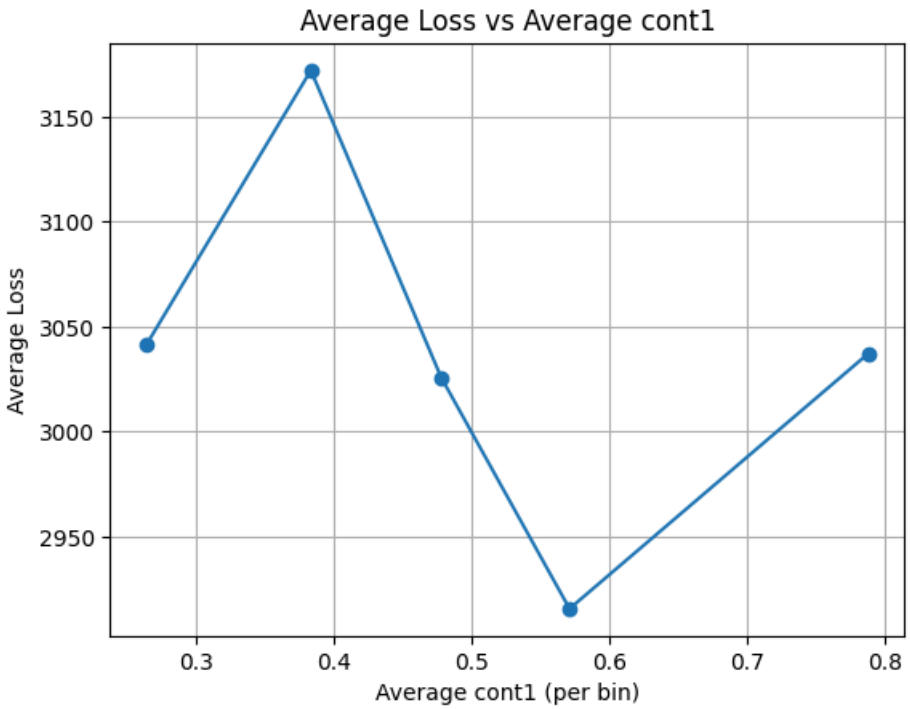
Here we are checking category 2. We can identify that this has a inverse relation, but given it is a little less vast in its differences, we as a group decided to use category 1. This leads us to our first major assumption/drawback, we can only looks at one categorical column, and one group in our column (in this case A)

In [11]:
```
# testing cont1 first, looking for a smooth mean across variables for a better risk class continuous variable
# Create bins
df['cont1_bin'] = pd.qcut(df['cont1'], q=5)

# Group and include mean of cont1 inside each bin
group_cont1 = df.groupby('cont1_bin').agg({
    'loss': ['count', 'mean', 'median', 'std'],
    'cont1': 'mean'   # <-- this is what you want
})
print(group_cont1)

# Flatten column names
group_cont1.columns = ['count', 'mean_loss', 'median_loss', 'std_loss', 'mean_cont1']

# Plot using mean cont1 as x-axis
plt.plot(group_cont1['mean_cont1'], group_cont1['mean_loss'], marker='o')

plt.xlabel('Average cont1 (per bin)')
plt.ylabel('Average Loss')
plt.grid(True)
plt.title('Average Loss vs Average cont1')
```

```
                    loss                                        cont1
                   count         mean    median          std      mean
cont1_bin
(-0.000984, 0.325]  39417  3041.310499  2201.920  2767.542138  0.264195
(0.325, 0.453]      36758  3172.163238  2161.225  3048.726005  0.383554
(0.453, 0.502]      37185  3025.874428  2080.580  2888.545870  0.478015
(0.502, 0.643]      38492  2915.612804  2015.485  2867.756737  0.571106
(0.643, 0.985]      36466  3037.315235  2129.120  2946.667476  0.787928
```

```
/tmp/ipykernel_5587/4066324401.py:6: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
  group_cont1 = df.groupby('cont1_bin').agg({
```

Out[11]: Text(0.5, 1.0, 'Average Loss vs Average cont1')

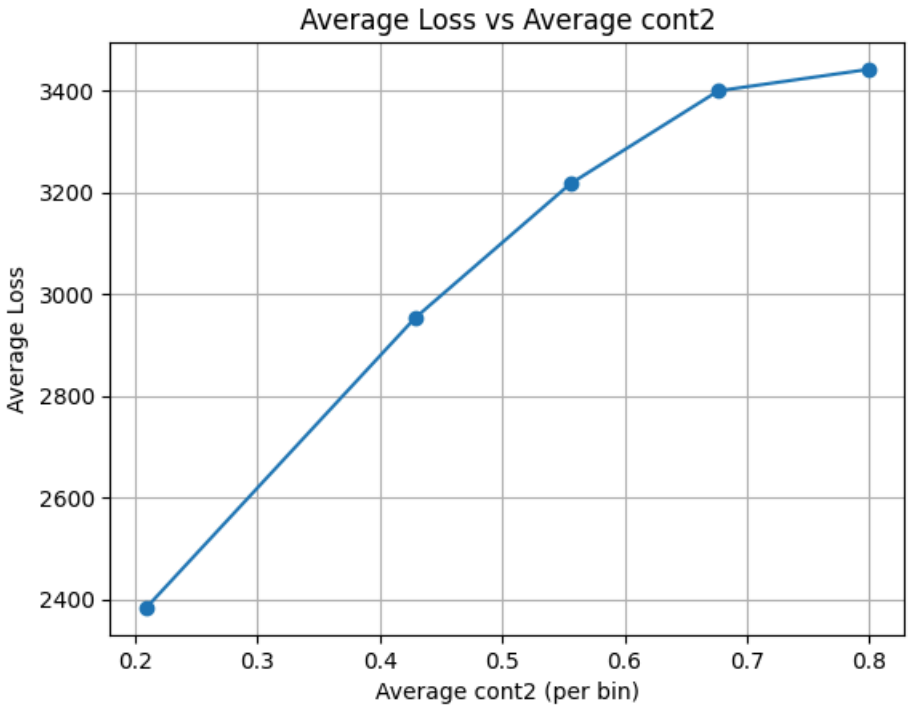


No clear trend from cont1. For risk classes we need a consistent increase or decrease in mean loss through the 5 bins

In [12]:
```
#same analyze for cont2
df['cont2_bin'] = pd.qcut(df['cont2'], q=5)  # cutting cont2 into 5 bins

#print summary
group_cont2 = df.groupby('cont2_bin')['loss'].agg(['count', 'mean', 'median', 'std'])
print(group_cont2)

#compute mean
mean_cont2 = df.groupby('cont2_bin')['cont2'].mean()

# Plot using actual averages
plt.plot(mean_cont2, group_cont2['mean'], marker='o')

plt.xlabel('Average cont2 (per bin)')
plt.ylabel('Average Loss')
plt.title('Average Loss vs Average cont2')
plt.grid(True)
plt.show()
```

```
                                count         mean    median          std
cont2_bin
(0.00014999999999999996, 0.299]  42258  2383.556433  1899.910  1789.039929
(0.299, 0.489]                   50040  2952.783606  2177.130  2462.708271
(0.489, 0.556]                   20975  3216.650567  2221.060  3004.055635
(0.556, 0.737]                   52362  3399.022636  2238.605  3412.660474
(0.737, 0.863]                   22683  3441.117577  2110.290  3760.474812
```

```
/tmp/ipykernel_5587/4128185608.py:5: FutureWarning: The default of observed=False is deprecated and will be changed to True in
a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and si
lence this warning.
  group_cont2 = df.groupby('cont2_bin')['loss'].agg(['count', 'mean', 'median', 'std'])
/tmp/ipykernel_5587/4128185608.py:9: FutureWarning: The default of observed=False is deprecated and will be changed to True in
a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and si
lence this warning.
  mean_cont2 = df.groupby('cont2_bin')['cont2'].mean()
```



Cont2 had a very distinct and predictable shape, so it makes the most sense to apply our function(spline) over this continuous value. Notice, we could of picked any continous value (with some restrictions), but given this has a clear trend, it makes it a perfect continuous variable to use for our spline.

Now we know for our project scope:

$x =$ continuous variable (2) binned

$y =$ losses

GIven we have 5 risk bins we have an four cubic spline pieces to deal with

our orginal idea for this section was the following:

From probability we know premium is the following

$$Premium_i = \lambda_i \cdot E[X]$$

Where λ_i is a poisson random variable representing relative frequency across the five risk classes, and $E[X]$ is the average loss

If we can find our lamdba, we could multiple it by our spline to find $E[X]$

The following code and explanation shares why this is not possible, yet another feasiable solution was made...

In [13]:
```
# Assigning bins and variables to cont1
df['risk_class'] = pd.qcut(df['cont2'], q=5)
# Computing the frequency
frequency = df.groupby('risk_class')['loss'].apply(lambda x: (x > 0).mean())
# Computing severity (already found, but repeating here to group by risk class)
severity = df[df['loss'] > 0].groupby('risk_class')['loss'].mean()
```

```
/tmp/ipykernel_5587/1492300654.py:4: FutureWarning: The default of observed=False is deprecated and will be changed to True in
a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and si
lence this warning.
  frequency = df.groupby('risk_class')['loss'].apply(lambda x: (x > 0).mean())
/tmp/ipykernel_5587/1492300654.py:6: FutureWarning: The default of observed=False is deprecated and will be changed to True in
a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and si
lence this warning.
  severity = df[df['loss'] > 0].groupby('risk_class')['loss'].mean()
```

Important note: for this project, frequency has been estimated, since calculating frequency requires you know the number of claims in your dataset, as well as the exposure, which we don't know for this dataset. Exposure refers to how many units of risk we observed. This could mean the total number of insurance policies, the total number of insured cars, the total number of policy years, it depends on what you are looking for. Because we don't have the total number of anything, just the claims and the loss amounts and several categorical and continuous variables, we have to assume exposure.

In [14]:
```
group_cont2['count'] # tells us the total number of claims

exposure = 10000 #assumed exposure
group_cont2['lambda'] = group_cont2['count'] / exposure # each groups relative frequency
print(group_cont2[['count', 'lambda']])
```

```
                                 count  lambda
cont2_bin
(0.00014999999999999996, 0.299]  42258  4.2258
(0.299, 0.489]                   50040  5.0040
(0.489, 0.556]                   20975  2.0975
(0.556, 0.737]                   52362  5.2362
(0.737, 0.863]                   22683  2.2683
```

The lambda should be increasing with each bin, but its not.

After lots of research I have concluded that because this dataset only contains observed claims and no exposure information, we are not able to directly estimate frequency. To combat this, we will have to assume frequency is constant across each risk class.

This means that instead of modeling premium as frequency multiplied by severity, we will have to model premium as average loss in each bin, which we have already computed.

Step 3b: Computing the Cubic Spline for $E[X]$ given our Risk Scores

Creating our spline points:

In [15]:
```
group_cont2['premium'] = group_cont2['mean'] # calculates mean cost for each risk class

# points for interpolation
midpoints = [interval.mid for interval in group_cont2.index] # take the midpoint for interpolation instead of endpoints
premiums = group_cont2['mean'].values

#print points for interpolation
for i, (x, y) in enumerate(zip(midpoints, premiums), start=1):
  print(f"Point {i}: ({x:.8f}, {y:.8f})")
```

```
Point 1: (0.14957500, 2383.55643310)
Point 2: (0.39400000, 2952.78360552)
Point 3: (0.52250000, 3216.65056687)
Point 4: (0.64650000, 3399.02263645)
Point 5: (0.80000000, 3441.11757660)
```

In [16]:
```
#automating the text
x_list = [0.14957500, 0.39400000, 0.52250000, 0.64650000, 0.80000000]
y_list = [2383.55643310, 2952.78360552, 3216.65056687, 3399.02263645, 3441.11757660]

for i in range(len(x_list)):
  spline_str = '$E_'+str(i)+ ' = a_'+str(i)+' + b_'+str(i)+ '(x - '+str(x_list[i])+')^1 + c_'+str(i)+ '(x - '+str(x_list[i])+
  print(spline_str)
```

```
$E_0 = a_0 + b_0(x - 0.149575)^1 + c_0(x - 0.149575)^2 + d_0(x - 0.149575)^3$
$E_1 = a_1 + b_1(x - 0.394)^1 + c_1(x - 0.394)^2 + d_1(x - 0.394)^3$
$E_2 = a_2 + b_2(x - 0.5225)^1 + c_2(x - 0.5225)^2 + d_2(x - 0.5225)^3$
$E_3 = a_3 + b_3(x - 0.6465)^1 + c_3(x - 0.6465)^2 + d_3(x - 0.6465)^3$
$E_4 = a_4 + b_4(x - 0.8)^1 + c_4(x - 0.8)^2 + d_4(x - 0.8)^3$
```

Now that we have 5 spline points, we will have 4 spline pieces:

E_0, E_1, E_2, E_3

where $j = 0, 1, 2, 3$

and our points have became:

- $(x_0, y_0) = (0.14957500, 2383.55643310)$
- $(x_1, y_1) = (0.39400000, 2952.78360552)$
- $(x_2, y_2) = (0.52250000, 3216.65056687)$
- $(x_3, y_3) = (0.64650000, 3399.02263645)$
- $(x_4, y_4) = (0.80000000, 3441.11757660)$

Given us the following general equations:

- $E_0(x) = a_0 + b_0(x - 0.149575) + c_0(x - 0.149575)^2 + d_0(x - 0.149575)^3$
- $E_1(x) = a_1 + b_1(x - 0.394) + c_1(x - 0.394)^2 + d_1(x - 0.394)^3$
- $E_2(x) = a_2 + b_2(x - 0.5225) + c_2(x - 0.5225)^2 + d_2(x - 0.5225)^3$
- $E_3(x) = a_3 + b_3(x - 0.6465) + c_3(x - 0.6465)^2 + d_3(x - 0.6465)^3$

Become we our still in the world of probabilities, we are not concerned with what happens after the boundaries, so we are going to consider a Natural Spline

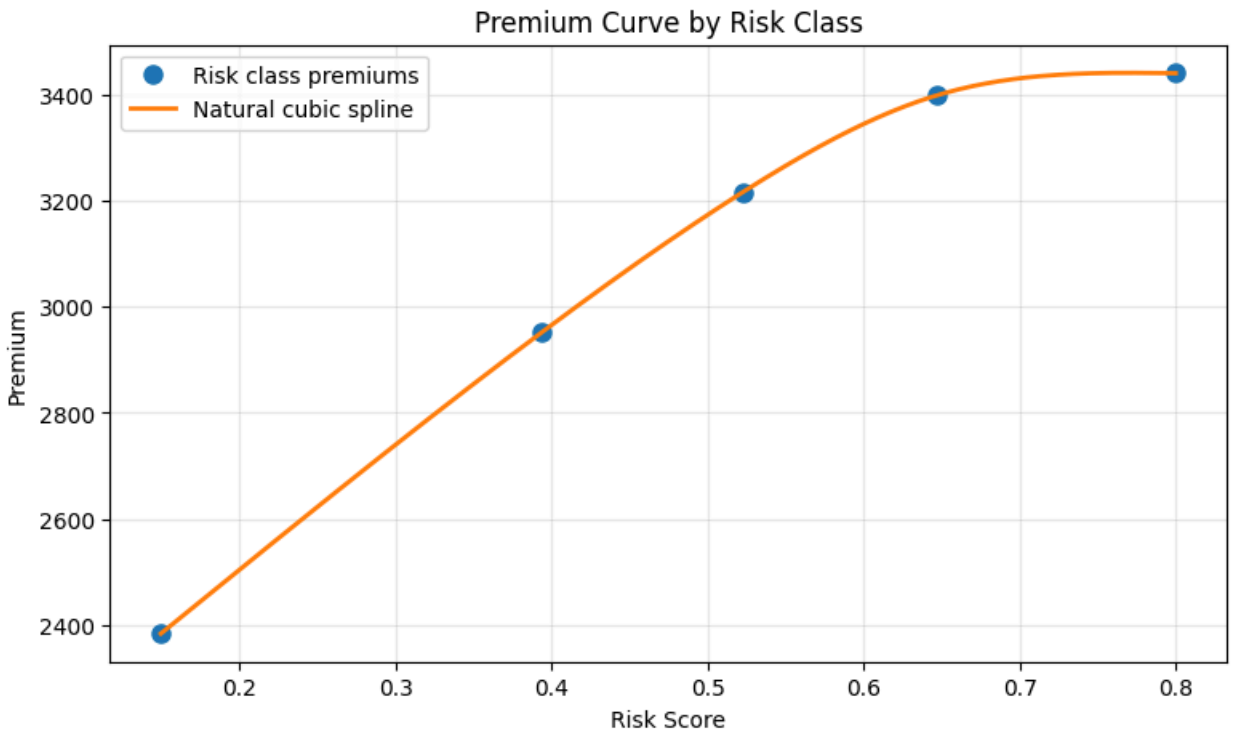
We could do this by hand or through python, and I believe it would be cleaner to do it using python.

For this interpolation, we will be using CubicSpline from scipy.interpolate

In [17]:
```
from scipy.interpolate import CubicSpline

# taking average loss in each bin
group_cont2 = df.groupby('cont2_bin')['loss'].agg(['count', 'mean', 'median', 'std'])
group_cont2 = group_cont2.copy()
group_cont2['premium'] = group_cont2['mean']

# bin midpoints and premiums
x = np.array([interval.mid for interval in group_cont2.index], dtype=float)
y = group_cont2['premium'].values

# printing points
points_table = pd.DataFrame({
    'Midpoint (x_i)': x,
    'Premium (y_i)': y
})

print("Cubic spline interpolation points:")
print(points_table.to_string(index=False))

# fitting the cubic spline
cs = CubicSpline(x, y, bc_type='natural') # bc_type natural tells python to use a natural cubic spline so that we can evaluat

x_smooth = np.linspace(x.min(), x.max(), 300)
y_smooth = cs(x_smooth)

#plotting
plt.figure(figsize=(9, 5))
plt.plot(x, y, 'o', markersize=8, label='Risk class premiums')
plt.plot(x_smooth, y_smooth, '-', linewidth=2, label='Natural cubic spline')
plt.title('Premium Curve by Risk Class')
plt.xlabel('Risk Score')
plt.ylabel('Premium')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

```
/tmp/ipykernel_5587/1288531846.py:4: FutureWarning: The default of observed=False is deprecated and will be changed to True in
a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and si
lence this warning.
  group_cont2 = df.groupby('cont2_bin')['loss'].agg(['count', 'mean', 'median', 'std'])
```

```
Cubic spline interpolation points:
 Midpoint (x_i)  Premium (y_i)
       0.149575    2383.556433
       0.394000    2952.783606
       0.522500    3216.650567
       0.646500    3399.022636
       0.800000    3441.117577
```



In [18]:

```python
#matrix of coefficients
import sympy
from sympy import Matrix
#Matrix form
mat1 = Matrix(cs.c)
#print(mat1)

#automate equations (same process I used before with some minor fixes (to amount of data))
for i in range(4):
  a = str(round(mat1[i,3],8))
  b = str(round(mat1[i,2],8))
  c = str(round(mat1[i,1],8))
  d = str(round(mat1[i,0],8))
  spline_str = '$E_'+str(i)+ ' = '+a+' + '+b+ '(x - '+str(x_list[i])+')^1 + '+c+ '(x - '+str(x_list[i])+')^2 + '+d+ '(x - '+s
  print(spline_str)
```

```
$E_0 = 13187.98875157 + -11578.40470136(x - 0.149575)^1 + -2496.50913179(x - 0.149575)^2 + -1095.76625535(x - 0.149575)^3$
$E_1 = -6073.06882010 + -1765.90227120(x - 0.394)^1 + -803.49800089(x - 0.394)^2 + 0.0(x - 0.394)^3$
$E_2 = 895.71151368 + 1867.74392900(x - 0.5225)^1 + 2197.91186396(x - 0.5225)^2 + 2394.30686283(x - 0.5225)^3$
$E_3 = 3399.02263645 + 3216.65056687(x - 0.6465)^1 + 2952.78360552(x - 0.6465)^2 + 2383.55643310(x - 0.6465)^3$
```

Complete Spline Equations

$$E_0 = 13187.98875157 - 11578.40470136(x - 0.149575) - 2496.50913179(x - 0.149575)^2 - 1095.76625535(x - 0.149575)^3$$

$$E_1 = -6073.06882010 - 1765.90227120(x - 0.394) - 803.49800089(x - 0.394)^2$$

$$E_2 = 895.71151368 + 1867.74392900(x - 0.5225) + 2197.91186396(x - 0.5225)^2 + 2394.30686283(x - 0.5225)^3$$

$$E_3 = 3399.02263645 + 3216.65056687(x - 0.6465) + 2952.78360552(x - 0.6465)^2 + 2383.55643310(x - 0.6465)^3$$

Numerical Intregation Using our Spline

Using our expected value spline we can preform numerical integration over our entire spline.

Because we are using a computer package, we will use the **Simpsons 1/3 Composite Rule** (found in our textbook). Which states...

$ \int_{a}^{b} f(x),dx \approx \frac{h}{3}\left[f(x_0)

- 4\sum_{i=1,, i\ \text{odd}}^{n-1} f(x_i)
- 2\sum_{i=2,, i\ \text{even}}^{n-2} f(x_i)
- f(x_n)

\right]$

where

$h = \frac{b-a}{n}, \quad n \text{ is even}$

Using this we can decide any amount of points to integrate over.

Where our $f(x)$ is one of the following splines we calculated. Therefore:

- $f(x) = E_0$ for the interval $(0.149575, 0.394)$
- $f(x) = E_1$ for the interval $(0.394, 0.5225)$
- $f(x) = E_2$ for the interval $(0.5225, 0.6465)$
- $f(x) = E_3$ for the interval $(0.6465, 0.8)$

Our bound are:

- $a = 0.149575$
- $b = 0.8$

Our step size for the computer simulation is:

- $h = 0.000650425$ provided $n = 1000$

To ensure this result becomes a expected we must divided by the following:

- difference of the bounds (b-a)

So the result becomes:

$E[X] = \frac{\int_a^b f(x)dx}{b-a}$

Typically in insurance and probability theory the expected value of a continuous variable is defined as the following:

$E[X] = \int_{x_0}^{x_{max}} xf(x)dx$ where x is the loss amount and f(x) is the probability density function. Because we do not have this exact setup, our method works just as well. (In a future section we will explore the numerical approimation useing the probability density function).

In [19]:
```
#Numerical Integration of our spline
#imports:
from scipy.stats import lognorm
from scipy.integrate import simpson

#the second approach we can try is the following:

#variables(from before)
mu = mu
sigma = float(root)

#use the same risk points to define our losses
x = np.array([interval.mid for interval in group_cont2.index], dtype=float)
risk_points = np.linspace(x.min(), x.max(), 1000)
y_values = cs(risk_points)
expected_premium = simpson(y, x) / (risk_points.max() - risk_points.min())
print('Expected Value Using Numerical Integration of Our Cubic Spline: ' + str(expected_premium))
```

```
Expected Value Using Numerical Integration of Our Cubic Spline: 3059.128839194696
```

Using our Cubic Spline our numerical approxiamtion for the expected value becomes:

$E[X] = 3059.128839194696$

Step 3c: Creating Cubic Splines based on Category

Typically in expected value calculations and premium creation in insurance are based on category. What this means is rates are typically adjusted depending on what catgory they fall into. For example, for health insurance, being a smoker verus a non smoker creates a high difference. To reflect this spliting idea, we are going to follow the same procedure as above, but create two cubic splines, one for each category. As we addressed in an earlier section, category (1) is an interesting case to look at due to the large discrepency between groups A and B. to achieve this the following steps need to be taken.

1. Split data amongst category 1
2. define risk scores(as we did before) for each group
3. create a cubic spline for each category
4. find the expected value by doing: $E[X] = P(A)E[X|A] + P(B)E[X|B]$
5. expected value solved through using Simpsons 1/3 Composite Method

Where A are those in group A, and B are those in group B

Statement (4) can be expanded to the following resul (in which we used for the one cubic spline approach):

$$E[X] = P(A)\int f(a) + P(B)\int f(b)$$

this approach is more sound to insurance principles

(*Note: for consistency we are still using continuous variable 2*)

In [27]:
```
#Find the probabilities:
df.groupby('cat1')['cat1'].count()

prob_a = df.groupby('cat1')['cat1'].count()['A'] / df.groupby('cat1')['cat1'].count().sum()
prob_b = df.groupby('cat1')['cat1'].count()['B']/df.groupby('cat1')['cat1'].count().sum()
print('Probability of A: ', prob_a)
print('Probability of B: ', prob_b)
```

```
Probability of A:  0.7516541169723553
Probability of B:  0.24834588302764474
```

$$P(A) = 0.7516541169723553$$

$$P(B) = 0.24834588302764474$$

Now we are going to build our Cubic Spline for group A. Again the approach is the exact same (define risk scores, and create a cubic spline off of that)

In [32]:
```
# define values for A
df_a = df[df['cat1'] == 'A'].loc[:,:]
df_a['cont2_bin'] = pd.qcut(df_a['cont2'], q=5)  # cutting cont2 into 5 bins

#print summary
group_cont2_a = df_a.groupby('cont2_bin')['loss'].agg(['count', 'mean', 'median', 'std'])
#print(group_cont2_a)

#compute mean
mean_cont2_a = df_a.groupby('cont2_bin')['cont2'].mean()

df_a['risk_class'] = pd.qcut(df_a['cont2'], q=5)
# Computing the frequency
frequency = df_a.groupby('risk_class')['loss'].apply(lambda x: (x > 0).mean())
# Computing severity (already found, but repeating here to group by risk class)
severity = df_a[df_a['loss'] > 0].groupby('risk_class')['loss'].mean()

group_cont2_a['premium'] = group_cont2_a['mean'] # calculates mean cost for each risk class

# points for interpolation
midpoints_a = [interval.mid for interval in group_cont2_a.index] # take the midpoint for interpolation instead of endpoints
premiums_a = group_cont2_a['mean'].values

#print points for interpolation
for i, (x, y) in enumerate(zip(midpoints_a, premiums_a), start=1):
  print(f"Point {i}: ({x:.8f}, {y:.8f})")
```

```
/tmp/ipykernel_5587/1387281923.py:6: FutureWarning: The default of observed=False is deprecated and will be changed to True in
a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and si
lence this warning.
  group_cont2_a = df_a.groupby('cont2_bin')['loss'].agg(['count', 'mean', 'median', 'std'])
/tmp/ipykernel_5587/1387281923.py:10: FutureWarning: The default of observed=False is deprecated and will be changed to True i
n a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and
silence this warning.
  mean_cont2_a = df_a.groupby('cont2_bin')['cont2'].mean()
/tmp/ipykernel_5587/1387281923.py:14: FutureWarning: The default of observed=False is deprecated and will be changed to True i
n a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and
silence this warning.
  frequency = df_a.groupby('risk_class')['loss'].apply(lambda x: (x > 0).mean())
Point 1: (0.14957500, 2591.29566785)
Point 2: (0.39400000, 3284.17326506)
Point 3: (0.52250000, 3617.68187836)
Point 4: (0.64650000, 3879.27991687)
Point 5: (0.80000000, 3975.89861160)
/tmp/ipykernel_5587/1387281923.py:16: FutureWarning: The default of observed=False is deprecated and will be changed to True i
n a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and
silence this warning.
  severity = df_a[df_a['loss'] > 0].groupby('risk_class')['loss'].mean()
```

In [34]:

```python
#automating the text
x_list = [0.14957500, 0.39400000, 0.52250000, 0.64650000, 0.80000000]
y_list = [2591.29566785, 3284.17326506, 3617.68187836, 3879.27991687, 3975.89861160]

for i in range(len(x_list)):
  spline_str = '$A_'+str(i)+ ' = a_'+str(i)+' + b_'+str(i)+ '(x - '+str(x_list[i])+')^1 + c_'+str(i)+ '(x - '+str(x_list[i])+
  print(spline_str)
```

```
$A_0 = a_0 + b_0(x - 0.149575)^1 + c_0(x - 0.149575)^2 + d_0(x - 0.149575)^3$
$A_1 = a_1 + b_1(x - 0.394)^1 + c_1(x - 0.394)^2 + d_1(x - 0.394)^3$
$A_2 = a_2 + b_2(x - 0.5225)^1 + c_2(x - 0.5225)^2 + d_2(x - 0.5225)^3$
$A_3 = a_3 + b_3(x - 0.6465)^1 + c_3(x - 0.6465)^2 + d_3(x - 0.6465)^3$
$A_4 = a_4 + b_4(x - 0.8)^1 + c_4(x - 0.8)^2 + d_4(x - 0.8)^3$
```

Let's Denote Our Splines using A (since that is the category we are in)

Therefore we have

A_0, A_1, A_2, A_3

where $j = 0, 1, 2, 3$

and our points have became:

- $(x_0, y_0) = (0.14957500, 2591.29566785)$
- $(x_1, y_1) = (0.39400000, 3284.17326506)$
- $(x_2, y_2) = (0.52250000, 3617.68187836)$
- $(x_3, y_3) = (0.64650000, 3879.27991687)$
- $(x_4, y_4) = (0.80000000, 3975.89861160)$

Given us the following general equations:

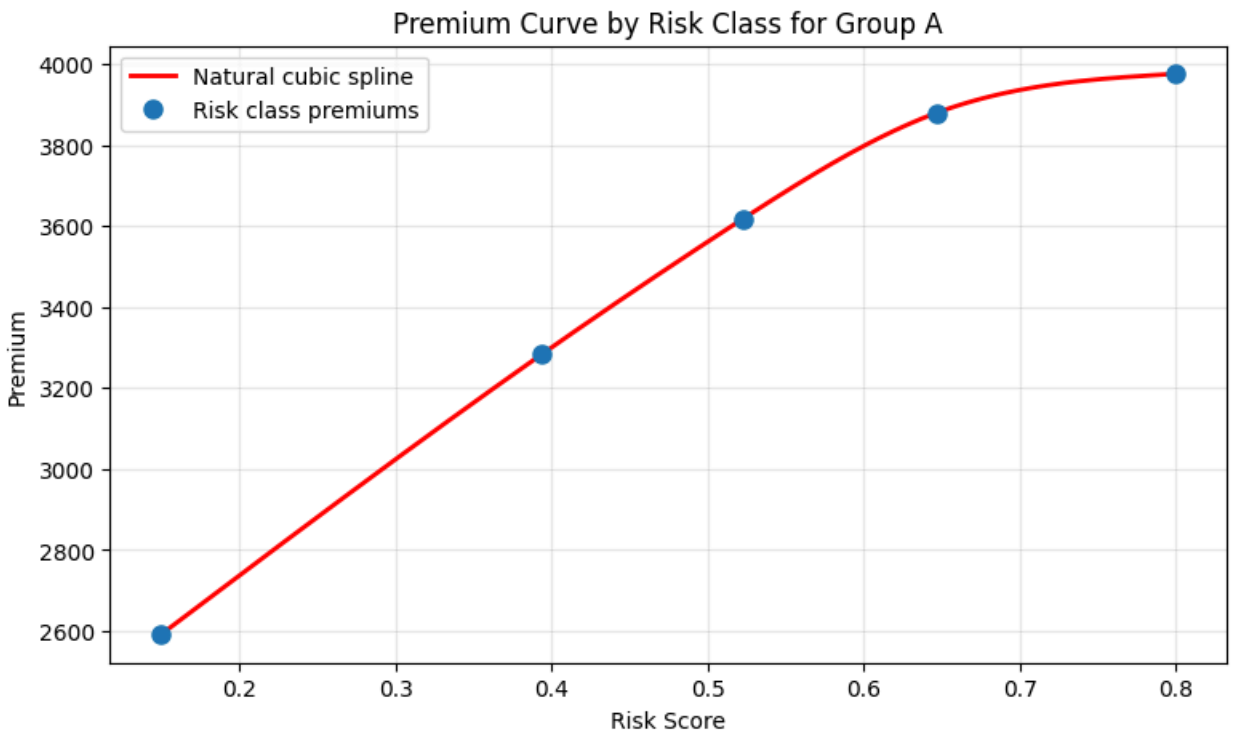
- $A_0(x) = a_0 + b_0(x - 0.149575) + c_0(x - 0.149575)^2 + d_0(x - 0.149575)^3$
- $A_1(x) = a_1 + b_1(x - 0.394) + c_1(x - 0.394)^2 + d_1(x - 0.394)^3$
- $A_2(x) = a_2 + b_2(x - 0.5225) + c_2(x - 0.5225)^2 + d_2(x - 0.5225)^3$
- $A_3(x) = a_3 + b_3(x - 0.6465) + c_3(x - 0.6465)^2 + d_3(x - 0.6465)^3$

In [40]:

```python
#creating the spline
x_a = np.array([interval.mid for interval in group_cont2_a.index], dtype=float)
y_a = group_cont2_a['premium'].values

# printing points
points_table = pd.DataFrame({
    'Midpoint (x_i)': x_a,
    'Premium (y_i)': y_a
})

print("Cubic spline interpolation points (A):")
print(points_table.to_string(index=False))
```

```python
# fitting the cubic spline
cs_a = CubicSpline(x_a, y_a, bc_type='natural')

x_smooth_a = np.linspace(x_a.min(), x_a.max(), 300)
y_smooth_a = cs_a(x_smooth_a)

#plotting
plt.figure(figsize=(9, 5))
plt.plot(x_smooth_a, y_smooth_a, '-', linewidth=2, label='Natural cubic spline', color = 'red')
plt.plot(x_a, y_a, 'o', markersize=8, label='Risk class premiums')
plt.title('Premium Curve by Risk Class for Group A')
plt.xlabel('Risk Score')
plt.ylabel('Premium')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

```
Cubic spline interpolation points (A):
 Midpoint (x_i)  Premium (y_i)
       0.149575    2591.295668
       0.394000    3284.173265
       0.522500    3617.681878
       0.646500    3879.279917
       0.800000    3975.898612
```



In [41]:

```python
#Matrix form
mata = Matrix(cs_a.c)
#print(mat1)

#automate equations (same process I used before with some minor fixes (to amount of data))
for i in range(4):
  a = str(round(mata[i,3],8))
  b = str(round(mata[i,2],8))
  c = str(round(mata[i,1],8))
  d = str(round(mata[i,0],8))
  spline_str = '$A_'+str(i)+ ' = '+a+' + '+b+ '(x - '+str(x_list[i])+')^1 + '+c+ '(x - '+str(x_list[i])+')^2 + '+d+ '(x - '+s
  print(spline_str)
```

```
$A_0 = 17011.17512915 + -19042.10421706(x - 0.149575)^1 + 216.44358327(x - 0.149575)^2 + -1136.57547247(x - 0.149575)^3$
$A_1 = -7833.64614697 + -749.98337822(x - 0.394)^1 + -833.42237957(x - 0.394)^2 + 0.0(x - 0.394)^3$
$A_2 = 1431.08087307 + 2495.45093419(x - 0.5225)^1 + 2698.91857407(x - 0.5225)^2 + 2902.62783920(x - 0.5225)^3$
$A_3 = 3879.27991687 + 3617.68187836(x - 0.6465)^1 + 3284.17326506(x - 0.6465)^2 + 2591.29566785(x - 0.6465)^3$
```

Complete Spline Equations for A

- $A_0 = 17011.17512915 - 19042.10421706(x - 0.149575) + 216.44358327(x - 0.149575)^2 - 1136.57547247(x - 0.149575)^3$

- $A_1 = -7833.64614697 - 749.98337822(x - 0.394) - 833.42237957(x - 0.394)^2$
- $A_2 = 1431.08087307 + 2495.45093419(x - 0.5225) + 2698.91857407(x - 0.5225)^2 + 2902.62783920(x - 0.5225)^3$
- $A_3 = 3879.27991687 + 3617.68187836(x - 0.6465) + 3284.17326506(x - 0.6465)^2 + 2591.29566785(x - 0.6465)^3$

Now we are going to build our Cubic Splines for group B.

In [42]:
```
# define values for B
df_b = df[df['cat1'] == 'B'].loc[:,:]
df_b['cont2_bin'] = pd.qcut(df_b['cont2'], q=5)  # cutting cont2 into 5 bins

#print summary
group_cont2_b = df_b.groupby('cont2_bin')['loss'].agg(['count', 'mean', 'median', 'std'])
#print(group_cont2_b)

#compute mean
mean_cont2_b = df_b.groupby('cont2_bin')['cont2'].mean()

df_b['risk_class'] = pd.qcut(df_b['cont2'], q=5)
# Computing the frequency
frequency = df_b.groupby('risk_class')['loss'].apply(lambda x: (x > 0).mean())
# Computing severity (already found, but repeating here to group by risk class)
severity = df_b[df_b['loss'] > 0].groupby('risk_class')['loss'].mean()

group_cont2_b['premium'] = group_cont2_b['mean'] # calculates mean cost for each risk class

# points for interpolation
midpoints_b = [interval.mid for interval in group_cont2_b.index] # take the midpoint for interpolation instead of endpoints
premiums_b = group_cont2_b['mean'].values

#print points for interpolation
for i, (x, y) in enumerate(zip(midpoints_b, premiums_b), start=1):
  print(f"Point {i}: ({x:.8f}, {y:.8f})")
```

```
Point 1: (0.15028500, 1715.18343413)
Point 2: (0.39400000, 1902.73886288)
Point 3: (0.55500000, 2007.07246701)
Point 4: (0.67900000, 1987.81222902)
Point 5: (0.80000000, 2013.67955354)
```

```
/tmp/ipykernel_5587/1541707850.py:6: FutureWarning: The default of observed=False is deprecated and will be changed to True in
a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and si
lence this warning.
  group_cont2_b = df_b.groupby('cont2_bin')['loss'].agg(['count', 'mean', 'median', 'std'])
/tmp/ipykernel_5587/1541707850.py:10: FutureWarning: The default of observed=False is deprecated and will be changed to True i
n a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and
silence this warning.
  mean_cont2_b = df_b.groupby('cont2_bin')['cont2'].mean()
/tmp/ipykernel_5587/1541707850.py:14: FutureWarning: The default of observed=False is deprecated and will be changed to True i
n a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and
silence this warning.
  frequency = df_b.groupby('risk_class')['loss'].apply(lambda x: (x > 0).mean())
/tmp/ipykernel_5587/1541707850.py:16: FutureWarning: The default of observed=False is deprecated and will be changed to True i
n a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and
silence this warning.
  severity = df_b[df_b['loss'] > 0].groupby('risk_class')['loss'].mean()
```

In [33]:
```
# define values for B
df_b = df[df['cat1'] == 'B'].loc[:,:]
df_b['cont2_bin'] = pd.qcut(df_b['cont2'], q=5)  # cutting cont2 into 5 bins

#print summary
group_cont2_b = df_b.groupby('cont2_bin')['loss'].agg(['count', 'mean', 'median', 'std'])
#print(group_cont2_b)

#compute mean
mean_cont2_b = df_b.groupby('cont2_bin')['cont2'].mean()

df_b['risk_class'] = pd.qcut(df_b['cont2'], q=5)
# Computing the frequency
frequency = df_b.groupby('risk_class')['loss'].apply(lambda x: (x > 0).mean())
# Computing severity (already found, but repeating here to group by risk class)
severity = df_b[df_b['loss'] > 0].groupby('risk_class')['loss'].mean()
```

```
group_cont2_b['premium'] = group_cont2_b['mean'] # calculates mean cost for each risk class

# points for interpolation
midpoints_b = [interval.mid for interval in group_cont2_b.index] # take the midpoint for interpolation instead of endpoints
premiums_b = group_cont2_b['mean'].values

#print points for interpolation
for i, (x, y) in enumerate(zip(midpoints_b, premiums_b), start=1):
  print(f"Point {i}: ({x:.8f}, {y:.8f})")
```

```
Point 1: (0.15028500, 1715.18343413)
Point 2: (0.39400000, 1902.73886288)
Point 3: (0.55500000, 2007.07246701)
Point 4: (0.67900000, 1987.81222902)
Point 5: (0.80000000, 2013.67955354)
```

```
/tmp/ipykernel_5587/1541707850.py:6: FutureWarning: The default of observed=False is deprecated and will be changed to True in
a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and si
lence this warning.
  group_cont2_b = df_b.groupby('cont2_bin')['loss'].agg(['count', 'mean', 'median', 'std'])
/tmp/ipykernel_5587/1541707850.py:10: FutureWarning: The default of observed=False is deprecated and will be changed to True i
n a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and
silence this warning.
  mean_cont2_b = df_b.groupby('cont2_bin')['cont2'].mean()
/tmp/ipykernel_5587/1541707850.py:14: FutureWarning: The default of observed=False is deprecated and will be changed to True i
n a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and
silence this warning.
  frequency = df_b.groupby('risk_class')['loss'].apply(lambda x: (x > 0).mean())
/tmp/ipykernel_5587/1541707850.py:16: FutureWarning: The default of observed=False is deprecated and will be changed to True i
n a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and
silence this warning.
  severity = df_b[df_b['loss'] > 0].groupby('risk_class')['loss'].mean()
```

In [43]:

```
#automating the text
x_list = [0.14957500, 0.39400000, 0.52250000, 0.64650000, 0.80000000]
y_list = [1715.18343413, 1902.73886288, 2007.07246701, 1987.81222902, 2013.67955354]

for i in range(len(x_list)):
  spline_str = '$B_'+str(i)+ ' = a_'+str(i)+' + b_'+str(i)+ '(x - '+str(x_list[i])+')^1 + c_'+str(i)+ '(x - '+str(x_list[i])+
  print(spline_str)
```

```
$B_0 = a_0 + b_0(x - 0.149575)^1 + c_0(x - 0.149575)^2 + d_0(x - 0.149575)^3$
$B_1 = a_1 + b_1(x - 0.394)^1 + c_1(x - 0.394)^2 + d_1(x - 0.394)^3$
$B_2 = a_2 + b_2(x - 0.5225)^1 + c_2(x - 0.5225)^2 + d_2(x - 0.5225)^3$
$B_3 = a_3 + b_3(x - 0.6465)^1 + c_3(x - 0.6465)^2 + d_3(x - 0.6465)^3$
$B_4 = a_4 + b_4(x - 0.8)^1 + c_4(x - 0.8)^2 + d_4(x - 0.8)^3$
```

Let's Denote Our Splines using B (since that is the category we are in)

Therefore we have

B_0, B_1, B_2, B_3

where $j = 0, 1, 2, 3$

and our points have became:

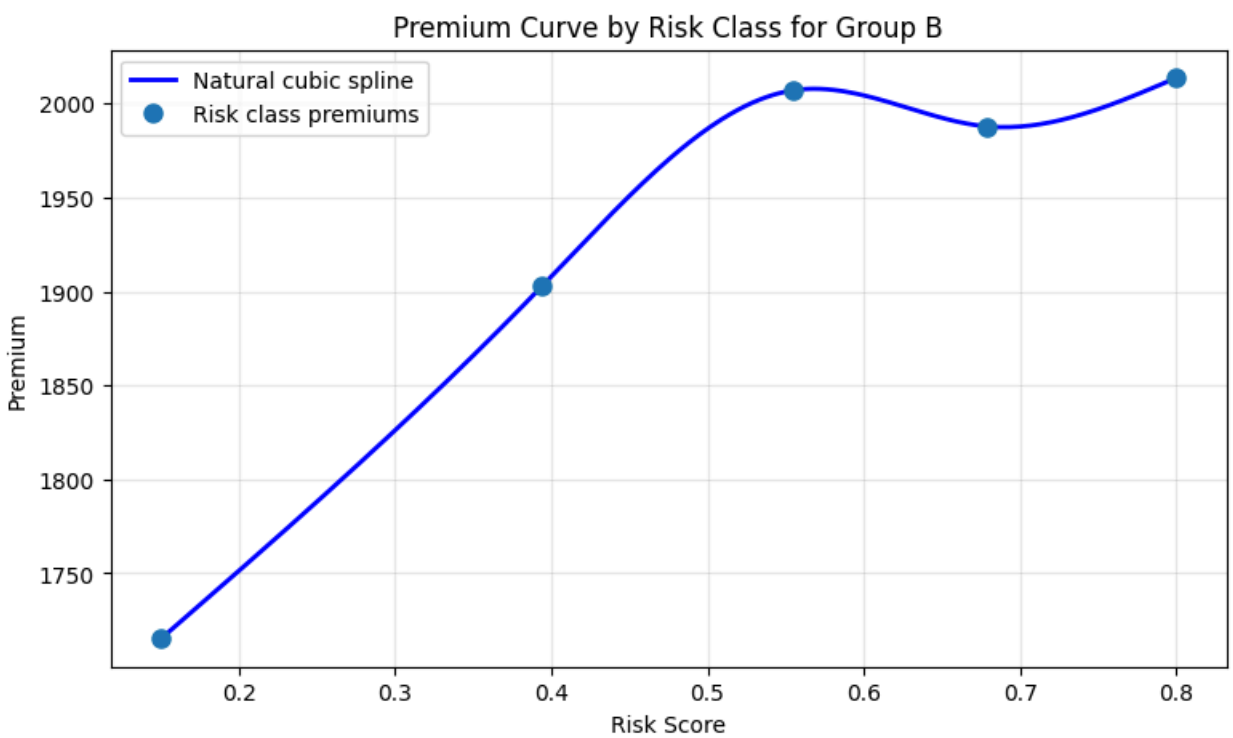
- $(x_0, y_0) = (0.14957500, 1715.18343413)$
- $(x_1, y_1) = (0.39400000, 1902.73886288)$
- $(x_2, y_2) = (0.52250000, 2007.07246701)$
- $(x_3, y_3) = (0.64650000, 1987.81222902)$
- $(x_4, y_4) = (0.80000000, 2013.67955354)$

Given us the following general equations:

- $B_0(x) = a_0 + b_0(x - 0.149575) + c_0(x - 0.149575)^2 + d_0(x - 0.149575)^3$
- $B_1(x) = a_1 + b_1(x - 0.394) + c_1(x - 0.394)^2 + d_1(x - 0.394)^3$
- $B_2(x) = a_2 + b_2(x - 0.5225) + c_2(x - 0.5225)^2 + d_2(x - 0.5225)^3$
- $B_3(x) = a_3 + b_3(x - 0.6465) + c_3(x - 0.6465)^2 + d_3(x - 0.6465)^3$

In [44]:
```
#creating the spline
x_b = np.array([interval.mid for interval in group_cont2_b.index], dtype=float)
y_b = group_cont2_b['premium'].values

# printing points
points_table = pd.DataFrame({
    'Midpoint (x_i)': x_b,
    'Premium (y_i)': y_b
})

print("Cubic spline interpolation points (A):")
print(points_table.to_string(index=False))

# fitting the cubic spline
cs_b = CubicSpline(x_b, y_b, bc_type='natural')

x_smooth_b = np.linspace(x_b.min(), x_b.max(), 300)
y_smooth_b = cs_b(x_smooth_b)

#plotting
plt.figure(figsize=(9, 5))
plt.plot(x_smooth_b, y_smooth_b, '-', linewidth=2, label='Natural cubic spline', color = 'Blue')
plt.plot(x_b, y_b, 'o', markersize=8, label='Risk class premiums')
plt.title('Premium Curve by Risk Class for Group B')
plt.xlabel('Risk Score')
plt.ylabel('Premium')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

```
Cubic spline interpolation points (A):
 Midpoint (x_i)  Premium (y_i)
       0.150285    1715.183434
       0.394000    1902.738863
       0.555000    2007.072467
       0.679000    1987.812229
       0.800000    2013.679554
```



In [45]:
```
#Matrix form
matb = Matrix(cs_b.c)
#print(mat1)

#automate equations (same process I used before with some minor fixes (to amount of data))
for i in range(4):
  a = str(round(matb[i,3],8))
  b = str(round(matb[i,2],8))
```

```
c = str(round(matb[i,1],8))
d = str(round(matb[i,0],8))
spline_str = '$B_'+str(i)+ ' = '+a+' + '+b+ '(x - '+str(x_list[i])+')^1 + '+c+ '(x - '+str(x_list[i])+')^2 + '+d+ '(x - '+s
print(spline_str)
```

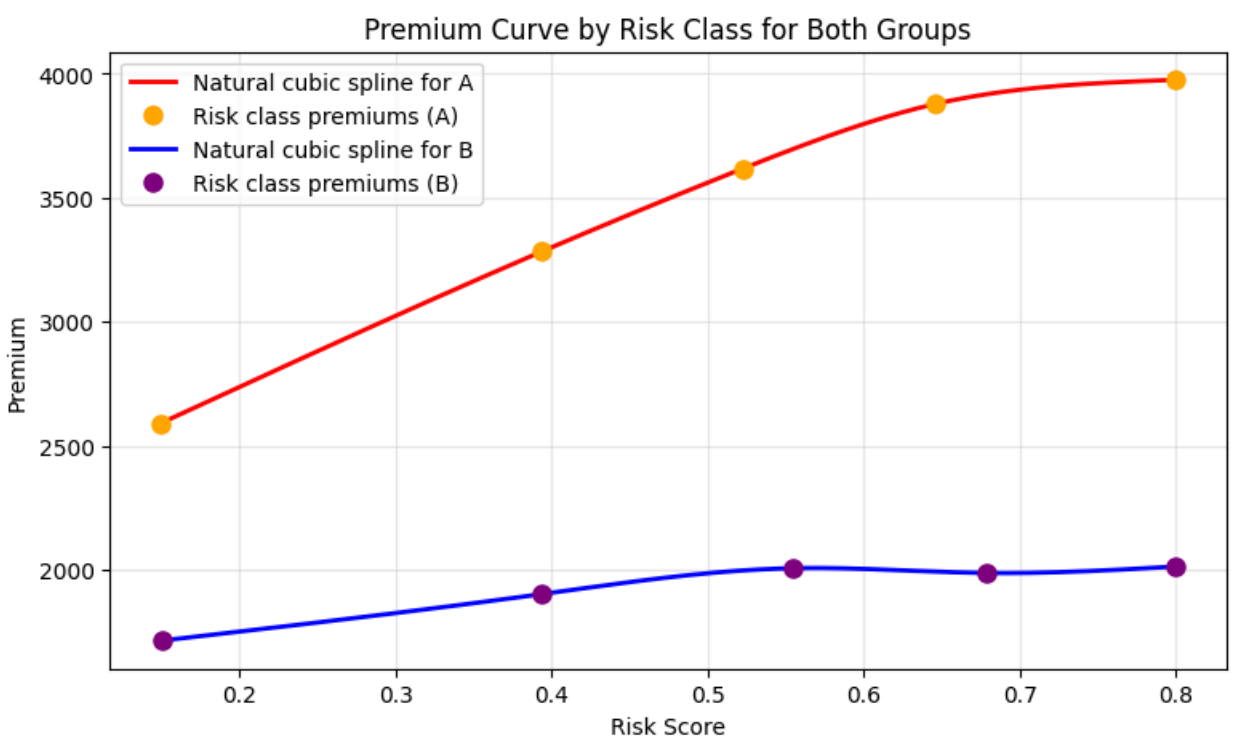
```
$B_0 = -9827.77612903 + 23480.79854014(x - 0.149575)^1 + -11893.89357131(x - 0.149575)^2 + 789.68778129(x - 0.149575)^3$
$B_1 = 3567.48273484 + -5167.37432209(x - 0.394)^1 + 577.37627285(x - 0.394)^2 + 0.0(x - 0.394)^3$
$B_2 = -73.99739914 + 124.38915768(x - 0.5225)^1 + 863.37884361(x - 0.5225)^2 + 722.66358528(x - 0.5225)^3$
$B_3 = 1987.81222902 + 2007.07246701(x - 0.6465)^1 + 1902.73886288(x - 0.6465)^2 + 1715.18343413(x - 0.6465)^3$
```

Complete Spline Equations for A

- $B_0 = -9827.77612903 + 23480.79854014(x - 0.149575) - 11893.89357131(x - 0.149575)^2 + 789.68778129(x - 0.149575)^3$
- $B_1 = 3567.48273484 - 5167.37432209(x - 0.394) + 577.37627285(x - 0.394)^2$
- $B_2 = -73.99739914 + 124.38915768(x - 0.5225) + 863.37884361(x - 0.5225)^2 + 722.66358528(x - 0.5225)^3$
- $B_3 = 1987.81222902 + 2007.07246701(x - 0.6465) + 1902.73886288(x - 0.6465)^2 + 1715.18343413(x - 0.6465)^3$

The following is the graph of both of the splines A and B:

In [48]:
```
#plotting
plt.figure(figsize=(9, 5))
plt.plot(x_smooth_a, y_smooth_a, '-', linewidth=2, label='Natural cubic spline for A', color = 'Red')
plt.plot(x_a, y_a, 'o', markersize=8, label='Risk class premiums (A)', color='orange')
plt.plot(x_smooth_b, y_smooth_b, '-', linewidth=2, label='Natural cubic spline for B', color = 'Blue')
plt.plot(x_b, y_b, 'o', markersize=8, label='Risk class premiums (B)', color='purple')
plt.title('Premium Curve by Risk Class for Both Groups')
plt.xlabel('Risk Score')
plt.ylabel('Premium')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



Complete Spline Equations for Both Groups Together

$A:$

- $A_0 = 17011.17512915 - 19042.10421706(x - 0.149575) + 216.44358327(x - 0.149575)^2 - 1136.57547247(x - 0.149575)^3$
- $A_1 = -7833.64614697 - 749.98337822(x - 0.394) - 833.42237957(x - 0.394)^2$
- $A_2 = 1431.08087307 + 2495.45093419(x - 0.5225) + 2698.91857407(x - 0.5225)^2 + 2902.62783920(x - 0.5225)^3$

- $A_3 = 3879.27991687 + 3617.68187836(x - 0.6465) + 3284.17326506(x - 0.6465)^2 + 2591.29566785(x - 0.6465)^3$

B :

- $B_0 = -9827.77612903 + 23480.79854014(x - 0.149575) - 11893.89357131(x - 0.149575)^2 + 789.68778129(x - 0.149575)^3$
- $B_1 = 3567.48273484 - 5167.37432209(x - 0.394) + 577.37627285(x - 0.394)^2$
- $B_2 = -73.99739914 + 124.38915768(x - 0.5225) + 863.37884361(x - 0.5225)^2 + 722.66358528(x - 0.5225)^3$
- $B_3 = 1987.81222902 + 2007.07246701(x - 0.6465) + 1902.73886288(x - 0.6465)^2 + 1715.18343413(x - 0.6465)^3$

Numerical Intregation Using our Splines

We will use the **Simpsons 1/3 Composite Rule** (found in our textbook) again. Which states...

$ \int_{a}^{b} f(x),dx \approx \frac{h}{3}\left[f(x_0)

- 4\sum_{i=1,, i\ \text{odd}}^{n-1} f(x_i)
- 2\sum_{i=2,, i\ \text{even}}^{n-2} f(x_i)
- f(x_n)

\right]$

where

$h = \frac{b-a}{n}, \quad n \text{ is even}$

Using this we can decide any amount of points to integrate over.

Recall, though we need to do this twice, once for $f(a)$, and once for $f(b)$: our new splines

For each Spline set are we use the following:

- $f(a), f(b) = A_0, B_0$ for the interval $(0.149575, 0.394)$
- $f(a), f(b) = A_1, B_1$ for the interval $(0.394, 0.5225)$
- $f(a), f(b) = A_2, B_2$ for the interval $(0.5225, 0.6465)$
- $f(a), f(b) = A_3, B_3$ for the interval $(0.6465, 0.8)$

Our bound are:

- $c = 0.149575$
- $d = 0.8$

Our step size for the computer simulation is:

- $h = 0.000650425$ provided $n = 1000$

To ensure this result becomes a expected we must divided by the following:

- difference of the bounds (d-c)

So the finalresult becomes:

$E[X] = P(A)\frac{\int_c^d A(x)dx}{d-c} + P(B)\frac{\int_c^d B(x)dx}{d-c}$

In [51]:
```
#Numerical Integration Work for own split cases:
x_a = np.array([interval.mid for interval in group_cont2_a.index], dtype=float)
risk_points_a = np.linspace(x_a.min(), x_a.max(), 1000)
y_values_a = cs_a(risk_points)
expected_premium_a = (simpson(y_a, x_a) / (risk_points_a.max() - risk_points_a.min())) * prob_a
print('Prob * Expected Value for A: ' + str(expected_premium_a))

x_b = np.array([interval.mid for interval in group_cont2_b.index], dtype=float)
risk_points_b = np.linspace(x_b.min(), x_b.max(), 1000)
y_values_b = cs_b(risk_points_b)
expected_premium_b = (simpson(y_b, x_b) / (risk_points_b.max() - risk_points_b.min())) * prob_b
```

```
print('Prob * Expected Value for B: ' + str(expected_premium_b))

print('Total Expected Value: ' + str(expected_premium_a + expected_premium_b))
```

```
Prob * Expected Value for A: 2583.8802515154284
Prob * Expected Value for B: 476.0370288785286
Total Expected Value: 3059.917280393957
```

Our split case value then becomes: $E[X] = 3059.917280393957$, which is very similar to our one case example (meaning we have a solid estimate)

Breaking this down further:

$P(A)E[X|A] = 2583.8802515154284$

and

$P(B)E[X|B] = 2583.8802515154284$

Step 4: Numerical Integration Using the Probability Density Formula

In the last section of our project, we spoke about general probability theory and its' approach to expected value. Recall:

$E[X] = \int_{x_0}^{x_{max}} xf(x)dx$

where x is the loss value, and $f(x)$ is the probability density function.

From our data analysis, and transformation section (step 1) we understand that we are dealing with a lognormal distribution. It's probability density function is defined as follows:

$f(x) = \frac{1}{x\sigma\sqrt{2\pi}} e^{-\frac{(lnx-\mu)^2}{2\sigma^2}}, x > 0$, or the loss must be positive

recall σ is the standard deviation, or more importantly the root we found from our Newton's Method.

$root = \sigma = 0.8118659387150824$

Given this is a very complex density function, numerical integration makes sense here. Our approach is going to be very similar to before. Again we are going to use **Simpson's 1/3 Composite Rule** where in this case our $f(x)$ is actually $x * f(x)$

Again we set $n = 1000$

In [20]:
```
#for sake of comparison, lets use our complex density function to integrate
#define our density function from the above function (in the write up)
density = lambda x: (1 / (x * sigma * np.sqrt(2*np.pi))) * np.exp(-(np.log(x) - mu)**2 / (2*sigma**2))
#data is our losses
loss_cols = ['cont2', 'loss']
loss_df = df[loss_cols].groupby('cont2')['loss'].mean()
loss_points = np.linspace(1e-6, df['loss'].max(), 1000)

#define our integrand
integrand = loss_points * density(loss_points)
#integrate
expected_premium_density = simpson(integrand, loss_points)
print('Expected Value Using Numerical Integration Using Loss Values and Density: ' + str(expected_premium_density))
```

```
Expected Value Using Numerical Integration Using Loss Values and Density: 3025.437751892531
```

So our expected value of our loss given this approach (density/probability approach) = 3025.437751892531

To ensure a proper/accurate result we now will vary the value of n to check if the inegral converges.

In [21]:
```
#showing that the solution converges
for n in [50, 100, 500, 1000, 2000, 3000, 4000, 5000, 6000, 7000, 8000, 9000, 10000]:
    loss_points = np.linspace(1e-6, df['loss'].max(), n)
    integrand = loss_points * density(loss_points)
    approx = simpson(integrand, loss_points)
    print(str(n) +':  ' + str(approx))
```

```
50:  3117.4411103165908
100:  3082.075277978928
500:  3025.039202144953
1000:  3025.437751892531
2000:  3025.4454120414625
3000:  3025.445010573515
4000:  3025.444964814266
5000:  3025.44495895969
6000:  3025.444958151344
7000:  3025.444958062231
8000:  3025.444958071324
9000:  3025.4449580860396
10000:  3025.4449580955716
```

Given these results,

$E[X] \approx 3025.4449580$

Step 5: Comparing the Methods

Since all of our methods worked toward the same goal (finding the expected loss), it is vital that we compare them and check accuracy.

Comparison Table

Method	Description	Estimated $E[X]$
Newton + Lognormal	Parametric (MLE-based)	3025.49
Spline + Simpson	Nonparametric (data-driven)	3059.13
Split Spline Model + Simpson	Category-adjusted spline	3059.92
Density Integration	Lognormal PDF integration	3025.44

The results show that all methods produce remarkably consistent estimates of the expected loss, with values falling within a very tight window: 3025 to 3060.

The Newton-based lognormal estimate is nearly identical to the density integration result, with an extremely small relative error. This is expected, since both methods rely on the same lognormal distribution (our Newton's approach justs finds the parameters). On the other hand, our density integration evaluates the expected value directly from the distribution. This distribution is very commonly used in the insurance world.

In contrast, the spline-based methods produce slightly higher estimates, with relative errors around 1% (the density was our basis given this is the theory based method). These differences arise because spline models are fully data-driven and do not impose a theory distribution, allowing them to capture the noise in the data (outliers...).

The close connection across all methods suggests that the expected loss estimate is solid to the choice of numerical technique. However, the small uptick for the Spline is important to note.

Next, we are going to look at the error involved in each estimate. Because in practice, the approach is usually the density based approach (step 4) we are going to use that as our base point for estimation.

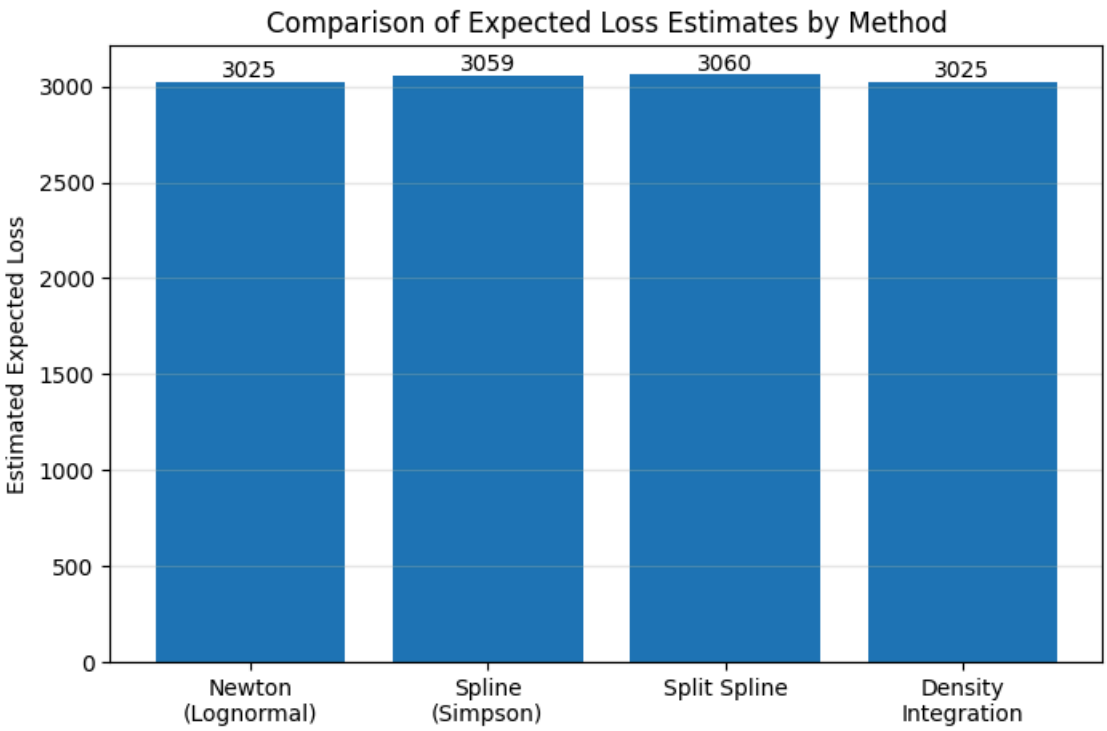
Error Comparison (Reference = Density Integration)

Method	Absolute Error	Relative Error
Newton + Lognormal	0.05	0.002%
Spline + Simpson	33.69	1.11%
Split Spline Model	34.48	1.14%

Comparison Graph(s):

In [59]:
```
#Bar chart of methods
methods = [
    "Newton\n(Lognormal)",
    "Spline\n(Simpson)",
```

```
    "Split Spline",
    "Density\nIntegration"
]

values = [3025.49, 3059.13, 3059.92, 3025.44]

plt.figure(figsize=(8,5))
plt.bar(methods, values)

plt.ylabel("Estimated Expected Loss")
plt.title("Comparison of Expected Loss Estimates by Method")

# annotate values on top
for i, v in enumerate(values):
    plt.text(i, v + 20, f"{v:.0f}", ha='center')

plt.grid(axis='y', alpha=0.3)
plt.show()
```



Assumptions

Methods:

- We assumed loss follows an **lognormal distribution** (this was because of the graph analyze we did at the beginning
- Binning of the continuous risk variable (cont2) does not significantly distort the underlying relationship between risk and expected loss.
- The midpoint of each bin is representative of all observations within that bin. (it is just a basis on use for our data)
- The cubic spline provides a smooth and reasonable approximation of the underlying loss function.
- Natural boundary conditions (second derivative equal to zero at endpoints) are appropriate for the spline model. recall we do not care for the behavior past our spline.
- Simpson's Rule provides an accurate approximation of the integral.

General:

- Observations are independent
- Exposure can be kept constant/ is constant

Drawbacks:

- The biggest draw back we experienced is the fact that the Allstate Claims Data was autonomized. While data is data (we still got what we needed from raw numbers), having labels would give us a better sense, and direction with the data.
- No exposure data: cannot properly model frequency (recall we had to make an adjustment to our spline approach)

- binning introduces information loss (but was needed for our Spline construction)
- Spline may overfit due to sparse nature of bins
- Lognormal assumption may misrepresent tail risk, we assumed based on shape of transformed data
- Simpson's rule introduces discrete errors (second piece of each equation)
- Using midpoints assumes we have a uniform in each bin (best way to reduce complexity though and allowed for a proper spline)

Method Drawback Table:

Method	Drawbacks
Newton + Lognormal	Very sensitive to extreme tail values, this work was based off an assumption
Spline + Simpson	Accuracy depends on our bins and number of points; spline may smooth out important 'jagged' natures in our graph; Simpson's Rule introduces discrete errors due to being a sum
Split Spline Model	Increased complexity but no significant improvement; possible overfitting within smaller subsets (group B more than group A)
Density Integration	Relies on lognormal assumption

Conclusion

Our project demonstrates the application of multiple numerical methods to approximate expected insurance loss. The results show that parametric based approaches (these are where we had to find μ, σ), such as the lognormal model estimated via Newtons Method, tend to produce higher estimates due to issues which arise for tail risk/ extreme cases.

In contrast, spline methods provide more stable, data-driven (purely from our data) approximations that are less influenced by outliers (tail risk we spoke of). Across all methods, the estimated (both sets of splines) expected loss falls within a relatively narrow range, indicating consistency even though our approach varied.

The comparison highlights a key tradeoff in numerical analysis: parametric (set function with parameters) models offer theoretical structure (general use in insurance probability theory), while spline methods methods provide flexibility and we found were more stable.

While these trade off are care, given we got to a similar result in each case, we can assume (to a high degree) that we have a proper estimate of the expected loss, given the variable(s) we were looking at.

All in all, this project represents a lot of the principles and methods insurance companies could use (and actually used) to look at expected loss. Like are project both the theory (parameter based methods), and model creation passed general probability models are used.

Reference Links

- https://towardsdatascience.com/log-normal-distribution-a-simple-explanation-7605864fb67c/
- https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.CubicSpline.html
- https://www.itl.nist.gov/div898/handbook/eda/section3/eda3669.htm
- https://mathworld.wolfram.com/LogNormalDistribution.html
- https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.newton.html
- https://www.kaggle.com/competitions/allstate-claims-severity
- https://balajichippada-20.medium.com/allstate-claims-severity-8ee0e4ecb2b5
- https://www.investopedia.com/terms/e/expected-loss-ratio-elr-method.asp